51. Design AI-Powered SOC (Security Operations Center)

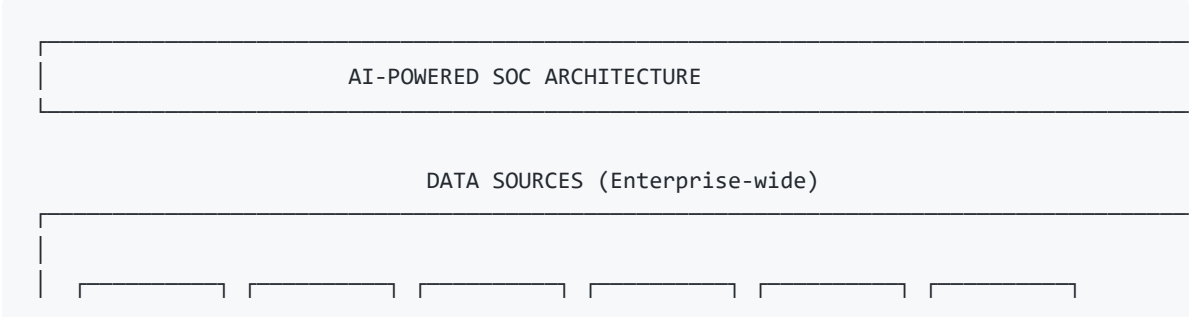
Problem Statement

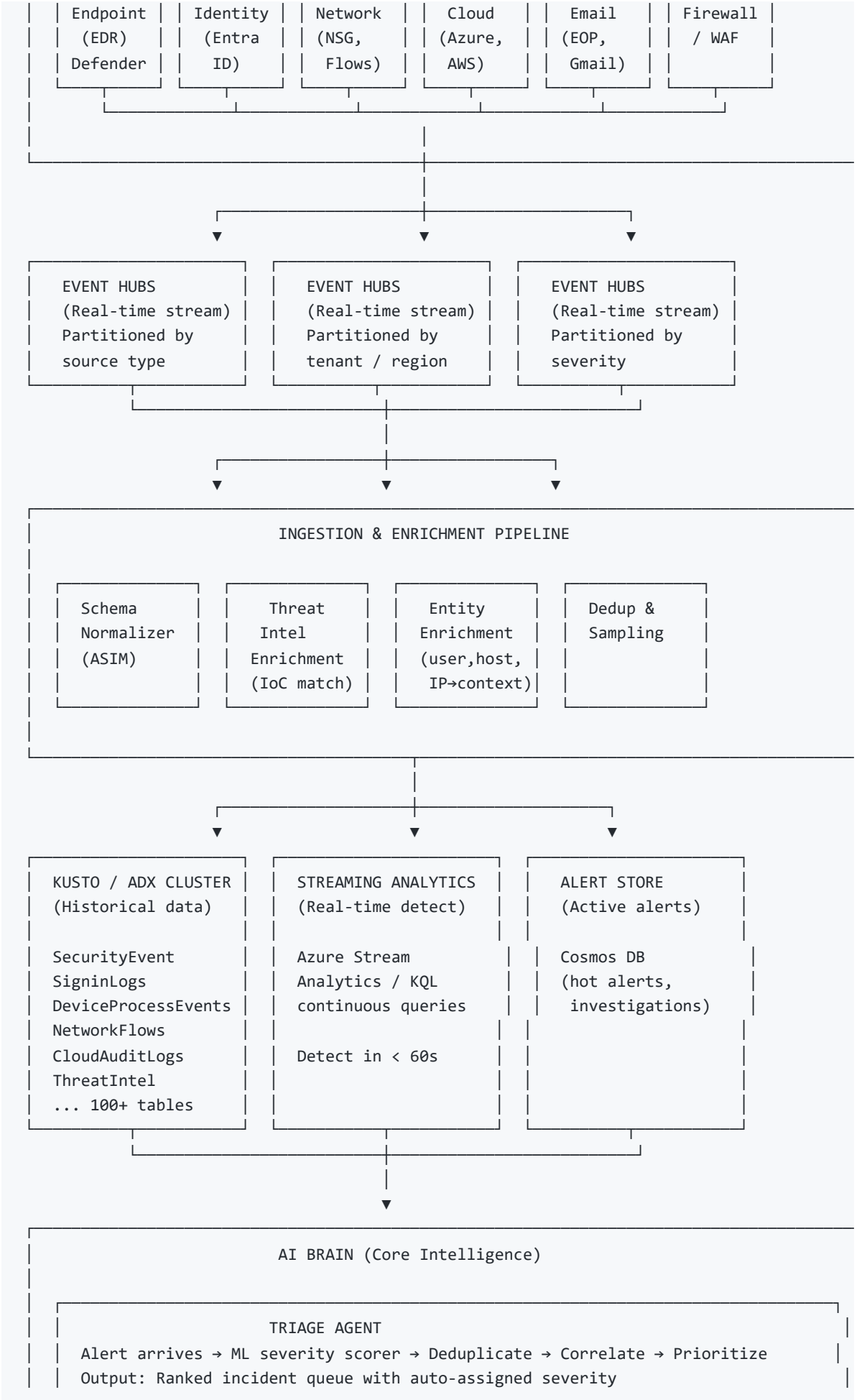
Design an AI-driven Security Operations Center platform that ingests massive volumes of security alerts, event logs, and telemetry from across an enterprise — stored in Kusto (Azure Data Explorer) tables and streaming via Event Hubs — and uses LLMs and ML models to autonomously triage, investigate, correlate, and respond to security incidents. The system should reduce analyst workload by 80%+ through automated investigation, natural language querying ("show me all lateral movement in the last 24h"), and AI-generated incident reports.

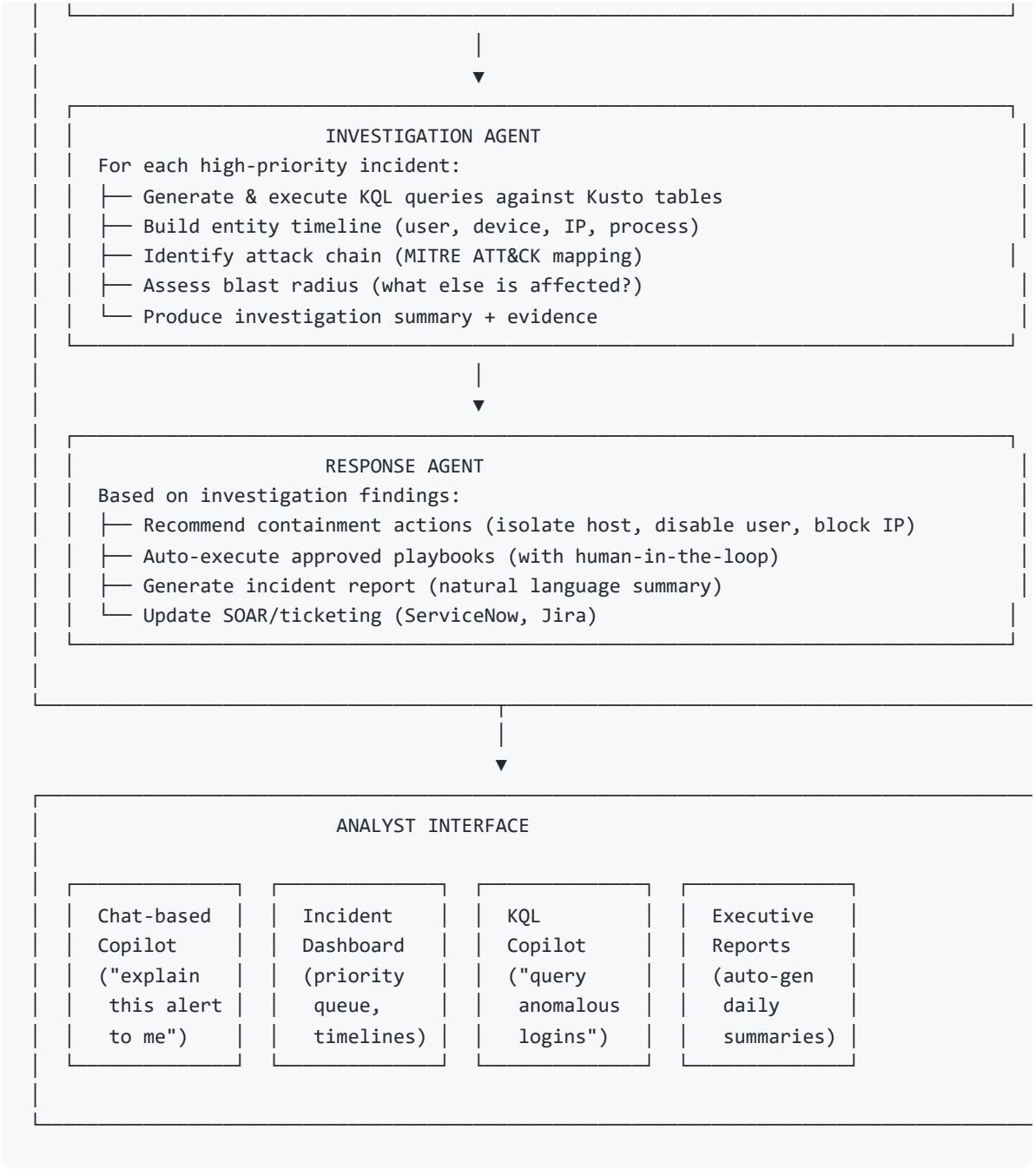
Clarifying Questions to Ask

1. **Data scale**: How many events/second ingested? (100K-1M+ EPS typical for large enterprise)
2. **Alert volume**: How many raw alerts/day? (tens of thousands, most are noise)
3. **Data sources**: Which log types? (endpoint, network, identity, cloud, email, firewall)
4. **Kusto schema**: Are the tables normalized (ASIM) or vendor-specific schemas?
5. **Response scope**: Recommend actions only, or auto-execute containment (isolate host, disable account)?
6. **Analyst interaction**: Chat-based investigation or automated playbooks or both?
7. **Multi-tenant**: Single enterprise SOC or MSSP serving multiple customers?
8. **Compliance**: Do we need full audit trail of every AI decision for regulatory review?
9. **Existing tooling**: Integrating into Microsoft Sentinel, or building from scratch?
10. **SLA**: What's the target MTTR (Mean Time to Respond) for critical incidents?

High-Level Design




Endpoint (EDR) Defender
Identity (Entra ID)
Network (NSG, Flows)
Cloud (Azure, AWS)
Email (EOP, Gmail)
Firewall / WAF
EVENT HUBS (Real-time stream) Partitioned by source type
EVENT HUBS (Real-time stream) Partitioned by tenant / region
EVENT HUBS (Real-time stream) Partitioned by severity
INGESTION & ENRICHMENT PIPELINE
Schema Normalizer (ASIM)
Threat Intel Enrichment (IoC match)
Entity Enrichment (user,host, IP→context)
Dedup & Sampling
KUSTO / ADX CLUSTER (Historical data)
SecurityEvent
SigninLogs
DeviceProcessEvents
NetworkFlows
CloudAuditLogs
ThreatIntel
... 100+ tables
STREAMING ANALYTICS (Real-time detect)
Azure Stream Analytics / KQL continuous queries
Detect in < 60s
ALERT STORE (Active alerts)
Cosmos DB (hot alerts, investigations)
AI BRAIN (Core Intelligence)
TRIAGE AGENT
Alert arrives → ML severity scorer → Deduplicate → Correlate → Prioritize
Output: Ranked incident queue with auto-assigned severity



API Design

Alert & Incident APIs

Method	Endpoint	Description
`POST`	`/v1/alerts/ingest`	Ingest a batch of raw alerts
`GET`	`/v1/incidents`	List incidents (filter by severity, status, time)

`GET`	`/v1/incidents/{id}`	Get incident details + AI investigation summary
`POST`	`/v1/incidents/{id}/investigate`	Trigger AI investigation on an incident
`POST`	`/v1/incidents/{id}/respond`	Execute a response action (with approval)
`PATCH`	`/v1/incidents/{id}`	Update incident status (analyst override)

AI Chat / Copilot APIs

Method	Endpoint	Description
`POST`	`/v1/copilot/chat`	Natural language security query (streaming SSE)
`POST`	`/v1/copilot/kql`	Generate KQL from natural language
`POST`	`/v1/copilot/explain`	Explain an alert, script, or command in plain English
`POST`	`/v1/copilot/summarize`	Summarize an incident for executive report

Query & Data APIs

Method	Endpoint	Description
`POST`	`/v1/kusto/query`	Execute KQL query against ADX cluster
`GET`	`/v1/entities/{type}/{id}/timeline`	Get entity timeline (user, device, IP)
`GET`	`/v1/entities/{type}/{id}/risk`	Get entity risk score and contributing factors
`GET`	`/v1/threat-intel/ioc/{value}`	Look up indicator of compromise

Playbook / Response APIs

Method	Endpoint	Description
`GET`	`/v1/playbooks`	List available response playbooks
`POST`	`/v1/playbooks/{id}/execute`	Execute playbook (requires approval for destructive actions)
`POST`	`/v1/actions/isolate-host`	Isolate a device from the network

`POST`	`/v1/actions/disable-user`	Disable an Entra ID account
`POST`	`/v1/actions/block-ip`	Add IP to firewall blocklist

Natural Language Chat Request

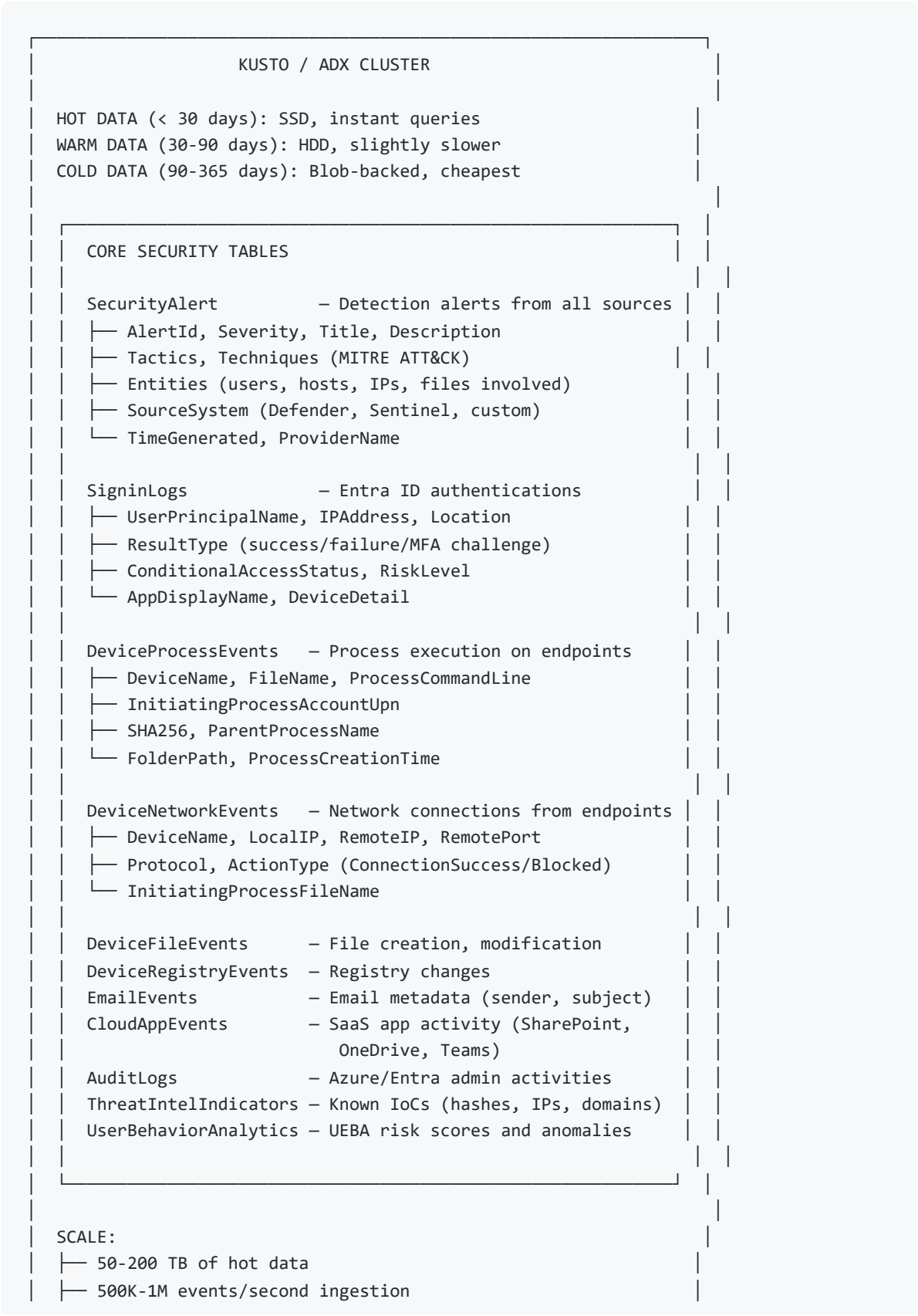
```
POST /v1/copilot/chat
{
  "session_id": "sess_soc_001",
  "message": "Show me all lateral movement activity for user jdoe@contoso.com in the
  "context": {
    "current_incident": "INC-2026-4521",
    "analyst_role": "Tier2",
    "tenant": "contoso"
  },
  "stream": true
}
```

Chat Response (Streamed)

```
{
  "session_id": "sess_soc_001",
  "steps": [
    {
      "type": "reasoning",
      "content": "I'll check RDP, SMB, PsExec, and WMI activity for this user across
    },
    {
      "type": "kql_query",
      "content": "DeviceNetworkEvents\n| where Timestamp > ago(48h)\n| where Initiat
      "query_id": "qry_abc123"
    },
    {
      "type": "kql_results",
      "content": "| RemoteIP | RemotePort | ConnectionCount | DistinctHosts |\n|----
    },
    {
      "type": "analysis",
      "content": "🔴 **Suspicious lateral movement detected.** User jdoe connected t
    },
    {
      "type": "recommendation",
      "content": "**Recommended actions:**\n1. Isolate device DESKTOP-JD01 (source o
    }
  ],
  "audit_trail_id": "audit_xyz789"
}
```

Data Layer: Kusto Tables & Event Hubs

Kusto Table Schema (ASIM-Normalized)



```
│  ├── 100+ tables across all data sources                          │
│  └── Sub-second query response for typical analyst queries        │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

Event Hubs Streaming Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    EVENT HUBS TOPOLOGY                           │
│                                                                  │
│  INGESTION HUBS (partitioned by source):                         │
│  ├── eh-endpoint-events     (32 partitions, 100K EPS)            │
│  ├── eh-identity-events     (16 partitions, 50K EPS)             │
│  ├── eh-network-events      (64 partitions, 500K EPS)            │
│  ├── eh-cloud-audit         (8 partitions, 10K EPS)              │
│  ├── eh-email-events        (8 partitions, 20K EPS)              │
│  └── eh-threat-intel        (4 partitions, 1K EPS)               │
│                                                                  │
│  CONSUMERS:                                                      │
│  ├── Consumer Group: "kusto-ingestion"                           │
│  │   └── ADX streaming ingestion (all events → Kusto tables)     │
│  ├── Consumer Group: "real-time-detection"                       │
│  │   └── Stream Analytics (continuous KQL detection rules)       │
│  ├── Consumer Group: "ai-triage"                                 │
│  │   └── AI Triage Agent (ML scoring + alert correlation)        │
│  └── Consumer Group: "enrichment"                                │
│      └── Enrichment pipeline (TI lookup, entity resolution)      │
│                                                                  │
│  ALERT OUTPUT HUB:                                               │
│  └── eh-alerts-enriched (triage agent writes scored alerts)      │
│      └── Consumed by: Investigation Agent, Dashboard, SOAR       │
│                                                                  │
│  RETENTION:                                                      │
│  ├── Event Hubs: 7 days (replay window for reprocessing)         │
│  └── Kusto: Hot 30d / Warm 90d / Cold 365d                       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

AI Brain: Three-Agent Architecture

Agent 1: Triage Agent (Real-Time, < 30 seconds)

```
┌──────────────────────────────────────────────────────────────────┐
│                    TRIAGE AGENT                                  │
│                                                                  │
│  INPUT: Raw alert from Event Hub                                 │
│                                                                  │
│  STEP 1: ML SEVERITY SCORING                                     │
```

```
│  ┌──────────────────────────────────────────────────────┐        │
│  │  Features:                                           │        │
│  │  ├── Alert source reputation (Defender = high signal) │        │
│  │  ├── Entity risk score (user/device UEBA score)      │        │
│  │  ├── Threat intel match (known IoC? = critical)      │        │
│  │  ├── Time of day (3 AM login = more suspicious)      │        │
│  │  ├── Historical false-positive rate for this rule    │        │
│  │  └── Asset criticality (domain controller > laptop)  │        │
│  │                                                      │        │
│  │  Output: true_positive_probability: 0.0 → 1.0        │        │
│  │  Model: XGBoost (fast, <5ms inference) or fine-tuned  │        │
│  │         classifier, retrained weekly on analyst      │        │
│  │         feedback (true positive / false positive)    │        │
│  └──────────────────────────────────────────────────────┘        │
│       │                                                          │
│       ▼                                                          │
│  STEP 2: ALERT CORRELATION & DEDUPLICATION                       │
│  ┌──────────────────────────────────────────────────────┐        │
│  │  Group alerts by:                                    │        │
│  │  ├── Same entity (user, device, IP) within time      │        │
│  │  │   window (15 min)                                 │        │
│  │  ├── Same MITRE technique across entities            │        │
│  │  │   (multiple hosts → campaign)                     │        │
│  │  └── Kill chain progression                          │        │
│  │      (recon → initial access → lateral movement)     │        │
│  │                                                      │        │
│  │  10,000 raw alerts/day → ~500 correlated incidents   │        │
│  └──────────────────────────────────────────────────────┘        │
│       │                                                          │
│       ▼                                                          │
│  STEP 3: PRIORITY QUEUE                                          │
│  ┌──────────────────────────────────────────────────────┐        │
│  │  Priority = severity_score × asset_criticality       │        │
│  │             × (1 + entity_risk) × recency_boost      │        │
│  │                                                      │        │
│  │  Queue: [Critical: 5] [High: 23] [Medium: 142]       │        │
│  │                                                      │        │
│  │  Auto-escalation: Critical incidents → immediate     │        │
│  │  Investigation Agent + page on-call analyst          │        │
│  └──────────────────────────────────────────────────────┘        │
│                                                                  │
│  OUTPUT: Correlated incident with severity, entities,            │
│          MITRE mapping, and priority score → Cosmos DB           │
└──────────────────────────────────────────────────────────────────┘
```

Agent 2: Investigation Agent (Deep Analysis, 30s-5min)

```
┌──────────────────────────────────────────────────────────────────┐
│                    INVESTIGATION AGENT                           │
│                                                                  │
│  INPUT: High-priority incident from Triage Agent                 │
│                                                                  │
│  AUTONOMOUS INVESTIGATION LOOP (ReAct pattern):                  │
```


ITERATION 1: Understand the alert
├── THINK: "Suspicious PowerShell on DESKTOP-JD01.
I need to check what commands were run and by whom."
├── ACT: Generate & execute KQL:
DeviceProcessEvents
| where DeviceName == "DESKTOP-JD01"
| where Timestamp > ago(1h)
| where FileName == "powershell.exe"
| project Timestamp, ProcessCommandLine, AccountName
└── OBSERVE: Found encoded PowerShell downloading
mimikatz from external IP 185.x.x.x
ITERATION 2: Check threat intel
├── THINK: "Need to check if 185.x.x.x is known bad"
├── ACT: Query ThreatIntelIndicators table +
external feeds (VirusTotal, AbuseIPDB)
└── OBSERVE: IP linked to APT29, flagged in 3 feeds
ITERATION 3: Assess lateral movement
├── THINK: "Mimikatz = credential theft. Check if
stolen creds were used to access other hosts"
├── ACT: Query SigninLogs + DeviceNetworkEvents for
jdoe@contoso.com → other hosts via RDP/SMB/WinRM
└── OBSERVE: jdoe connected to 3 additional servers
including domain controller DC01
ITERATION 4: Build attack timeline
├── THINK: "Map the full kill chain"
├── ACT: Correlate all findings into timeline
└── OBSERVE: Full chain:
14:32 - Phishing email received
14:35 - Malicious link clicked
14:36 - PowerShell downloads mimikatz
14:38 - Credential dumping (LSASS access)
14:42 - Lateral movement to 3 hosts
14:45 - Access to domain controller DC01
MITRE: T1566→T1059.001→T1003→T1021→T1078
OUTPUT:
├── Investigation summary (natural language)
├── Attack timeline with evidence links
├── MITRE ATT&CK mapping
├── Blast radius (affected entities)

```
│  ├── Confidence score (how sure is the AI?)                          │
│  ├── Recommended response actions                                    │
│  └── All KQL queries executed (for analyst review)                   │
│                                                                       │
│  TOOLS AVAILABLE TO AGENT:                                           │
│  ├── execute_kql(query, table) — run KQL against Kusto               │
│  ├── lookup_threat_intel(ioc) — check IoC databases                  │
│  ├── get_entity_risk(entity) — UEBA risk score                       │
│  ├── get_asset_info(host) — asset criticality, owner, OS             │
│  ├── get_user_baseline(upn) — normal behavior profile                │
│  ├── search_similar_incidents(description) — past incidents          │
│  └── decode_payload(encoded_string) — decode base64/PS               │
│                                                                       │
│  GUARDRAILS:                                                         │
│  ├── Max 20 KQL queries per investigation (cost control)             │
│  ├── Max 5-minute wall clock per investigation                       │
│  ├── Read-only access to Kusto (cannot modify data)                  │
│  ├── All queries logged to audit trail                               │
│  └── Escalate to human if confidence < 0.6                           │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

Agent 3: Response Agent (Action Execution)

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RESPONSE AGENT                                    │
│                                                                      │
│  INPUT: Investigation findings + recommended actions                 │
│                                                                      │
│  APPROVAL MODES:                                                     │
│  ┌──────────────────────────────────────────────────────┐            │
│  │  AUTO-APPROVE (low-risk, reversible actions):        │            │
│  │  ├── Block external IP at firewall                   │            │
│  │  ├── Quarantine email (move to junk)                 │            │
│  │  ├── Add IoC to threat intel blocklist               │            │
│  │  └── Create ServiceNow ticket                        │            │
│  │                                                      │            │
│  │  REQUIRE APPROVAL (high-impact actions):             │            │
│  │  ├── Isolate device from network                     │            │
│  │  ├── Disable user account                            │            │
│  │  ├── Force password reset                            │            │
│  │  ├── Wipe device                                     │            │
│  │  └── Revoke all active sessions                      │            │
│  │                                                      │            │
│  │  NEVER AUTO-EXECUTE:                                 │            │
│  │  ├── Delete data                                     │            │
│  │  ├── Modify firewall rules broadly                   │            │
│  │  └── Shut down production services                   │            │
│  └──────────────────────────────────────────────────────┘            │
│       │                                                              │
│       ▼                                                              │
│  EXECUTION:                                                          │
│  ├── Call Microsoft Graph API (disable user, revoke sessions)        │
│  ├── Call Defender API (isolate device, run AV scan)                 │
```

```
│  ├── Call Firewall API (block IP/domain)                          │
│  ├── Call SOAR webhook (trigger existing playbook)                │
│  └── Update incident status in Cosmos DB                          │
│       │                                                            │
│       ▼                                                            │
│  POST-ACTION:                                                      │
│  ├── Verify action succeeded (poll API for confirmation)          │
│  ├── Log action to audit trail (who approved, when, why)          │
│  ├── Generate incident report (PDF/Markdown)                      │
│  └── Notify analyst via Teams/email/PagerDuty                     │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

KQL Copilot: Natural Language → KQL

```
┌────────────────────────────────────────────────────────────────────┐
│                    KQL COPILOT                                     │
│                                                                    │
│  Analyst types: "Find all failed logins from outside the US       │
│                  for admin accounts in the last 7 days"           │
│       │                                                            │
│       ▼                                                            │
│  PROMPT ASSEMBLY:                                                  │
│  ┌──────────────────────────────────────────────────┐             │
│  │  System: You are a KQL expert for Microsoft Sentinel. │        │
│  │  Available tables: [SigninLogs, SecurityAlert, ...]   │        │
│  │  Table schemas: [columns, types, sample values]       │        │
│  │  Constraints: Read-only, max 10K results              │        │
│  │                                                       │        │
│  │  User: {analyst's natural language query}             │        │
│  └──────────────────────────────────────────────────┘             │
│       │                                                            │
│       ▼                                                            │
│  LLM GENERATES KQL:                                                │
│  ┌──────────────────────────────────────────────────┐             │
│  │  SigninLogs                                           │        │
│  │  | where TimeGenerated > ago(7d)                      │        │
│  │  | where ResultType != 0  // failed logins            │        │
│  │  | where Location !in ("US", "United States")         │        │
│  │  | where Identity has "admin"                         │        │
│  │  | summarize FailedAttempts = count(),                │        │
│  │              DistinctIPs = dcount(IPAddress)          │        │
│  │      by UserPrincipalName, Location                   │        │
│  │  | order by FailedAttempts desc                       │        │
│  └──────────────────────────────────────────────────┘             │
│       │                                                            │
│       ▼                                                            │
│  SAFETY CHECKS (before execution):                                 │
│  ├── Parse KQL AST → verify no mutations (.set, .drop)            │
│  ├── Check estimated cost (row scan estimate)                     │
│  ├── Add automatic | take 10000 if no limit present               │
```

```
│   └── Validate table/column names exist in schema                │
│       │                                                           │
│       ▼                                                           │
│   EXECUTE → Return results + AI interpretation                    │
│                                                                   │
│   FEEDBACK LOOP:                                                  │
│   ├── Analyst edits query → corrections stored for fine-tuning    │
│   ├── "This query was helpful" → positive signal                  │
│   └── Library of proven queries → few-shot examples for LLM       │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

Entity Risk Scoring (UEBA)

```
┌─────────────────────────────────────────────────────────────────┐
│                    ENTITY RISK ENGINE                           │
│                                                                 │
│   For each entity (user, device, IP), maintain a risk score:    │
│                                                                 │
│   RISK SIGNALS:                                                 │
│   ├── Anomalous login (new geo, impossible travel)        +15   │
│   ├── First-time access to sensitive resource             +10   │
│   ├── Unusual process execution on device                 +20   │
│   ├── Matching threat intelligence indicator              +40   │
│   ├── Multiple failed authentications                     +10   │
│   ├── Off-hours activity (outside normal pattern)         +5    │
│   ├── Privilege escalation attempt                        +25   │
│   └── Data exfiltration pattern (large upload)            +30   │
│                                                                 │
│   RISK DECAY: Score decays 10% per day with no new signals      │
│                                                                 │
│   BASELINE MODEL (per-entity):                                  │
│   ├── Normal login hours: 8 AM - 7 PM EST                       │
│   ├── Normal geolocations: [US-East, US-West]                   │
│   ├── Normal applications: [Outlook, Teams, SharePoint]         │
│   ├── Normal network patterns: Internal only                    │
│   └── Computed from 30-day rolling window via Kusto batch job   │
│                                                                 │
│   STORAGE:                                                      │
│   ├── Risk scores → Redis (real-time lookups, <1ms)             │
│   ├── Baselines → Kusto table (UserBehaviorAnalytics)           │
│   └── Risk history → Kusto (for trend analysis)                 │
│                                                                 │
│   EXAMPLE:                                                      │
│   Entity: jdoe@contoso.com                                      │
│   Risk Score: 87/100 (Critical)                                 │
│   Contributing factors:                                         │
│   ├── Login from Russia (never before)            +15           │
│   ├── Accessed domain controller (first time)     +10           │
│   ├── PowerShell encoded command execution        +20           │
│   ├── Mimikatz hash detected in process           +40           │
```

```
│   └── Baseline violation: 3 AM activity          +5             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Technology	Purpose
Event Ingestion	Azure Event Hubs (Kafka-compatible)	High-throughput streaming ingestion (1M+ EPS)
Data Store	Azure Data Explorer (Kusto)	Petabyte-scale log analytics, sub-second KQL queries
Stream Processing	Azure Stream Analytics / KQL streaming	Real-time detection rules (<60s latency)
Alert Store	Cosmos DB	Hot incident data, investigation state, agent threads
Risk Scores	Redis Cache	Real-time entity risk scores (<1ms lookup)
Triage Agent	XGBoost + correlation engine	ML severity scoring, alert dedup and grouping
Investigation Agent	LLM (GPT-4o) + KQL tools	Autonomous investigation via ReAct loop
Response Agent	LLM + Graph/Defender APIs	Recommend and execute containment actions
KQL Copilot	LLM + schema-aware prompt	Natural language → KQL translation
Threat Intelligence	Kusto table + external APIs	IoC matching (hashes, IPs, domains, URLs)
UEBA Engine	Kusto batch jobs + Redis	Per-entity behavioral baselines and anomaly detection
Audit Trail	Azure Immutable Blob Storage	Every AI decision, query, and action logged immutably
Notification	Azure Logic Apps / Event Grid	PagerDuty, Teams, email alerts for critical incidents

SOAR Integration	REST webhooks	Trigger existing playbooks in Sentinel/XSOAR/Splunk
Dashboard	React + Kusto Direct Query	Real-time incident queue, entity timelines, metrics

---

Scalability Considerations

1. Kusto Cluster Scaling

```
Challenge: 1M events/second, 100+ tables, analysts running ad-hoc queries simultaneously

Solutions:
├── Cluster auto-scale: Add/remove nodes based on CPU and ingestion lag
├── Hot/warm/cold tiering: SSD for recent data, HDD for older, Blob for archive
├── Materialized views: Pre-aggregate common queries (hourly alert counts, etc.)
├── Query limits: Per-user concurrent query cap, automatic timeout after 10 min
├── Follower databases: Read-only replicas for analyst queries (isolate from ingestion)
└── Partitioning: Partition large tables by TimeGenerated (most queries filter by time)

Sizing example (large enterprise):
├── 12-node cluster (D14v2: 16 cores, 112 GB RAM each)
├── Hot cache: 50 TB (30 days)
├── Warm storage: 200 TB (90 days)
├── Cold: 500 TB (365 days)
└── Ingestion: ~800K EPS sustained, 1.5M EPS burst
```

2. Event Hubs Scaling

```
Challenge: Bursty ingestion during incidents (2-5x normal volume)

Solutions:
├── Throughput Units: Auto-inflate from 20 → 100 TU during spikes
├── Partitioning: Partition by source type for parallel consumption
├── Consumer groups: Isolate Kusto ingestion from AI agents from dashboards
├── Capture: Auto-archive all events to Blob Storage (disaster recovery)
└── Kafka compatibility: Use Kafka consumer APIs for flexible processing

Partition strategy:
├── eh-endpoint-events: Partition by DeviceId (locality for per-device analysis)
├── eh-identity-events: Partition by TenantId (multi-tenant isolation)
└── eh-network-events: Partition by hash(SourceIP) (even distribution)
```

3. AI Agent Scaling

```
Challenge: Investigation agent uses expensive LLM calls + multiple KQL queries

Solutions:
├── Batch investigations: Group related incidents, investigate together
├── Caching: Cache KQL results for shared entities (same user, same host)
├── Model tiering: Use small model for triage, large model for investigation
├── Concurrency limit: Max 50 parallel investigations per cluster
├── Priority queue: Critical incidents get GPU priority over medium/low
└── Circuit breaker: If LLM API latency > 30s, queue and retry with backoff

Cost breakdown (per investigation):
├── KQL queries (5-10 queries): ~$0.01
├── LLM tokens (~8K input + 2K output): ~$0.10
├── Threat intel lookups: ~$0.001
└── Total: ~$0.12 per investigation
└── At 500 incidents/day: ~$60/day = ~$1,800/month
```

Trade-offs

Decision	Option A	Option B	Our Choice
Detection engine	Custom ML models only	LLM-based detection	**Hybrid** — ML for real-time triage, LLM for deep investigation
Alert storage	All in Kusto	Hot in Cosmos DB, cold in Kusto	**Hot in Cosmos DB** — faster CRUD for active incidents, cheaper at alert-level scale
KQL generation	Pre-built query templates	LLM-generated dynamic KQL	**LLM with safety checks** — more flexible, but parse AST before execution
Response automation	Fully autonomous	Human-in-the-loop only	**Tiered** — auto-execute low-risk, require approval for high-impact
Entity risk model	Rule-based scoring	ML anomaly detection	**Hybrid** — rules for known patterns + ML for novel anomalies
Investigation depth	Fixed playbook	Agent explores freely	**Agent with guardrails** — ReAct loop but max 20 queries, 5 min timeout

Multi-tenant data	Shared Kusto cluster	Cluster per tenant	**Shared with row-level security** — cost-efficient, enforce tenant isolation via query policies
Audit trail	Standard logging	Immutable ledger	**Immutable storage** — regulatory requirement for security decisions

---

Technologies Deep Dive

Data Platform

Technology	Purpose	Why
Azure Data Explorer (Kusto)	Log analytics & detection	Purpose-built for time-series security data, KQL is the query language of security
Azure Event Hubs	Streaming ingestion	Kafka-compatible, millions of events/sec, consumer groups for fan-out
Azure Stream Analytics	Real-time detection	Continuous KQL queries over streaming data, <60s detection latency
Cosmos DB	Active incident store	Low-latency CRUD for investigation state, TTL for auto-cleanup
Redis	Entity risk cache	Sub-millisecond risk score lookups during triage

AI & ML

Technology	Purpose	Why
GPT-4o (Azure OpenAI)	Investigation agent, KQL generation	Strong reasoning, tool-calling, KQL understanding
XGBoost / LightGBM	Triage severity scoring	Fast inference (<5ms), interpretable feature importance
Sentence Transformers	Alert similarity / dedup	Embed alert descriptions, cluster similar alerts

ONNX Runtime	ML model serving	Low-latency inference for triage scoring on CPU

Security & Compliance

Technology	Purpose	Why
Azure Immutable Blob Storage	Audit trail	Write-once, tamper-proof logs of every AI decision
Azure Key Vault	Secrets management	API keys for Defender, Graph, SOAR integrations
Managed Identity	Auth to Azure services	No stored credentials, automatic token rotation
Azure Private Link	Network isolation	All traffic stays on Microsoft backbone

---

How to Explain It Well

30-Second Pitch

> "We're building an AI SOC analyst that processes the massive stream of security events — 1M events/second flowing through Event Hubs into Kusto tables — and uses a three-agent architecture to autonomously handle incidents. The Triage Agent uses ML to score and correlate raw alerts, reducing 10,000 alerts/day down to ~500 actionable incidents. The Investigation Agent uses an LLM in a ReAct loop, autonomously generating and executing KQL queries to build attack timelines and map to MITRE ATT&CK. The Response Agent recommends and executes containment actions with tiered approval — auto-execute for low-risk, human approval for high-impact. Analysts interact via a KQL Copilot that translates natural language to KQL and a chat-based investigator."

Key Talking Points

1. **Data gravity in Kusto**: All security data lives in ADX — the AI agents query it directly via KQL, no data movement needed
2. **Three-agent separation**: Triage (fast ML, <30s) → Investigation (deep LLM, 30s-5min) → Response (action execution) — each with different latency and cost profiles
3. **KQL as the universal language**: Both human analysts and AI agents speak KQL — the LLM generates the same queries analysts would write

4. **Human-in-the-loop by design**: AI recommends, humans approve high-impact actions; full audit trail for every decision
5. **Alert fatigue is THE problem**: 10,000 alerts → 500 incidents → 50 requiring human attention. The AI handles the noise.

Whiteboard Sequence

1. Start with data sources → Event Hubs → Kusto (the data foundation)
2. Draw the three agents: Triage → Investigation → Response pipeline
3. Show the Investigation Agent's ReAct loop with KQL tool-calling
4. Add the KQL Copilot for analyst interaction
5. Overlay the governance layer (audit trail, approval gates)

Common Follow-up Questions

"How do you prevent the LLM from running dangerous KQL queries?"

- **Parse the KQL AST** before execution — reject any mutations (`.set`, `.drop`, `.delete`)
- **Read-only connection**: The Kusto principal used by the agent has `Viewer` role only
- **Row limits**: Automatically append `| take 10000` if no limit is present
- **Cost estimation**: Estimate data scan size from table stats, reject queries scanning >1TB
- **Query timeout**: Hard 2-minute timeout per KQL query
- **Audit logging**: Every generated query is logged with the LLM reasoning that produced it

"How do you handle false positives from the AI?"

- **Analyst feedback loop**: Every AI decision has "Confirm" / "False Positive" buttons
- **Weekly retraining**: Triage ML model retrained on analyst labels (true positive vs false positive)
- **Confidence scores**: Investigation Agent reports confidence; low confidence → escalate to human
- **Tunable thresholds**: Each organization adjusts auto-triage sensitivity per alert source
- **False positive suppression**: Repeated false positives auto-create suppression rules

"How does this scale for an MSSP (multi-tenant)?"

- **Kusto row-level security**: Each query automatically filtered by `TenantId` — no cross-tenant leakage
- **Event Hubs partitioning**: Partition by TenantId for isolation and parallel processing

- **Per-tenant AI config**: Different severity thresholds, approved response actions, and LLM personas
- **Cost attribution**: Token usage and KQL compute metered per tenant
- **Shared infrastructure**: Single Kusto cluster with tenant isolation — far cheaper than cluster-per-tenant

"How do you maintain the entity risk baselines at scale?"

- **Kusto scheduled jobs** (batch): Run daily aggregation queries over 30-day windows per entity
- **Materialized views**: Pre-compute `login_hour_distribution`, `geo_distribution`, `app_usage` per user
- **Redis snapshots**: Push updated risk scores to Redis every 15 minutes
- **Incremental updates**: Stream Analytics updates risk scores in near-real-time as events arrive
- **Cold start**: New entities start at baseline risk = 0, elevate based on first-seen signals

"What happens when the LLM API is down?"

- **Graceful degradation**:
  i. Triage Agent continues (ML model runs locally, no LLM dependency)
  ii. Investigation falls back to pre-built KQL playbook templates (no LLM reasoning)
  iii. Response Agent queues recommended actions for when LLM recovers
  iv. KQL Copilot shows "AI unavailable" — analyst writes KQL manually
- **Multi-region LLM**: Azure OpenAI deployed in 2+ regions with automatic failover
- **Queue-based architecture**: Incidents queue in Cosmos DB, processed when capacity returns